Lawrence Technological University
MRE 2024 – Unified Robotics I
Spring 2026

Final Project Report

Emma Harrison

Dr. Guarav Singh, Dr. Jose James

8 May 2026

Honor Code:

"I have neither given nor received unauthorized aid in completing this work, nor have I presented someone else's work as my own."

ABSTRACT

This project focuses on the application of computer vision, printed circuit board design, serial communication and microcontroller programming to the challenge of autonomous navigation. A small vehicle chassis equipped with two DC motors, a free-rotating support wheel, a 12V 7Ah lead-acid battery, and a custom printed circuit board is employed with the intention of navigating a simulated road course within a controlled laboratory environment. The goal of the project is that the vehicle maintains the requirement that navigation may never cause the vehicle to cross a white lane line, even during turns. In completing this project, the basic implementation requirements (related to both hardware and software) are explored and developed by students. Involved students gain a vital understanding of the requirements to successfully employ an autonomous navigation system using basic computer vision techniques.

Students began development by designing a custom printed circuit board (PCB). The PCB design is centered around the ATmega1284 microcontroller, with communication being facilitated by a Polulu AVR programmer implemented within a computer USB port. To implement dual motor control, the PCB employs the L298N dual H-bridge motor controller. All components used in the microcontroller board were hand-soldered and tested by the students during the PCB construction phase of the project. Students also implemented basic computer vision techniques in the C# programming language, utilizing the EMGU computer vision library. Using camera frame input obtained from the computer's integrated webcam, HSV filtering was applied and tuned to detect the position of the relevant lane markings within the webcam frame. Utilizing the yellow dashed lane divider for direct navigation and the red horizontal line for the stop mechanism, a simple bitmask was created to isolate pixels of the desired color within each captured camera frame. The lane-following logic implementation directly reflects a developed state machine with relevant inputs, outputs and state transitions. Finally, students implemented basic motor control and PCB operation using the C programming language, and the Microchip Studio platform. Using provided libraries designed to simplify motor control, students communicate serially from the computer vision program to the PCB. This allows for in-depth computation and identification to take place on the computer's CPU, and all motor control implementation takes place within the on-board PCB.

Development of this project led to successful implementation of the microcontroller and related code, as they were able to successfully operate the motors as desired. The ATmega1284 was able to successfully receive commands from the Windows Forms application code over serial communication using the COM20 port. The Windows Forms application code was successfully able to obtain live camera input from the integrated webcam, perform relevant HSV masking to detect particular shades of both yellow and red lane markings, and implement state machine logic to choose the robot action based on the immediate input. These results clearly reflect completion of the assignment specifications, and reflect the development of needed skills and knowledge to apply these techniques to future, more complex projects.

INTRODUCTION

To begin, this objective of this project is to successfully implement a robot capable of following established lane lines within a controlled environment for the purpose of completing the term assignment of the Lawrence Technological University Unified Robotics I course. The outer lane lines should never be crossed, and a red horizontal line should be used as both the start and end point of the robot's journey. An existing lane was used for implementation, within the controlled environment of an on-campus robotics laboratory. The lane designation is composed of both white outer lane markings, and a dashed yellow center lane marking. Using computer vision, camera input was processed to determine the position of the robot in relation to the lane markings. A student-constructed printed circuit board (PCB) equipped with an ATmega1284 microcontroller was used to provide appropriate power to the robotic platform's DC motors. To complete this task, line-following logic was implemented within the Windows Forms (.NET framework) platform, processing live image data using HSV filtering tuned to the exact hue, saturation and value of the lane markings used in-lab.

HARDWARE IMPLEMENTATION

To begin the process of implementing this project's hardware, students completed the development, design and construction of a custom printed circuit board. Under lab guidance, students implemented the EAGLE electronic design automation software to design a printed circuit board. Students were provided sample boards, a list of needed components, and necessary component libraries needed to successfully implement their design.

The PCB circuit schematic implemented in this project can be found in appendix A. This schematic diagram was produced using direct comparison to a provided schematic diagram, depicting the vital components to construct the circuit board. Students were also provided a list of needed components to purchase from electronics suppliers such as Amazon, Mouser, DigiKey, and Polulu. Table 1 lists all purchased components, and the supplier used to obtain the component.

Component Name	Supplier
ATmega1284 Microcontroller	DigiKey
USB AVR Programmer	Polulu
USB Cable A to Micro-B, 6 ft	Polulu
8x2 Character LCD Screen	Amazon
L298N Dual Full Bridge Motor Driver	Mouser
L298 Heat Sink	Mouser
MC78M05CTG 5V 0.5A Voltage Regulator	Mouser
Molex 39889-0042 2pos terminal block	Mouser
10k Ohm Trimmer Resistor	Mouser
2A, 60V Schottky Diode	Mouser
Momentary Tactile Switch	Mouser
Red LED	Mouser
10K Ohm Resistor	Mouser
330 Ohm Resistor	Mouser
150 Ohm Resistor	Mouser
0.1 µF Capacitor	Mouser
10 µF Capacitor	Mouser
18 pF Capacitor	Mouser
11.059 MHz Crystal	Mouser
40 Pin Socket	Mouser
1x8-Position Vertical Male Headers	Mouser
2x8-Position Vertical Female Headers	Mouser

Table 1. Summary of Purchased PCB Components and Suppliers

After identifying the source and specific components available for the project, students were able to determine factors such as the size of each resistor, the specific layout of multi-pin headers, and ultimately customize their circuit design to fit their specific needs. Once all component identifications were complete, the PCB circuit design was converted into a board layout file. Students further made use of the EAGLE software program to orient and position all desired components on the actual board. Figure 1 depicts the board layout schematic generated using the EAGLE program.

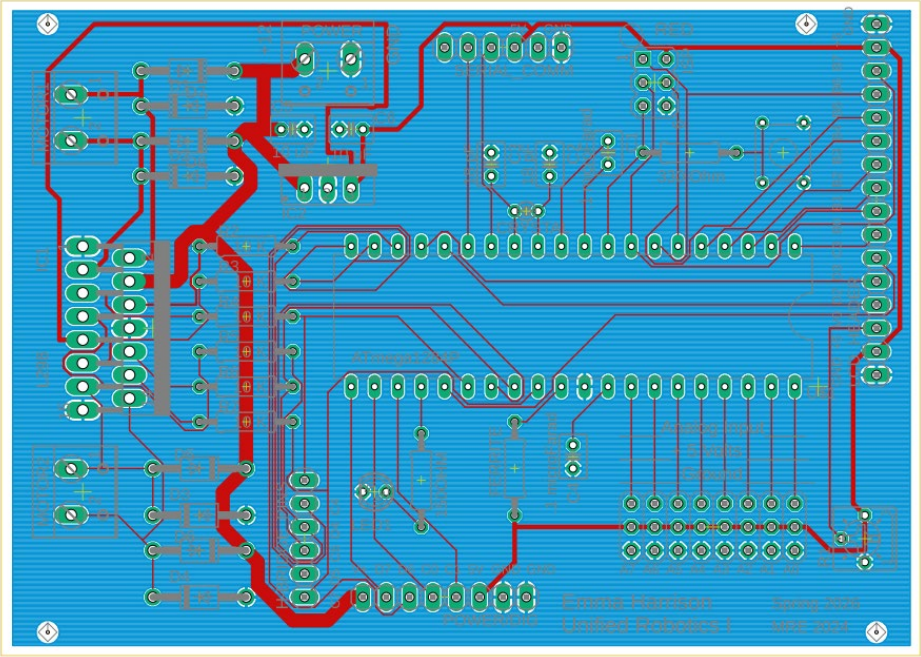


Figure 1. PCB Board Layout Generated by EAGLE Software

In completing this, each component was purposefully placed within the board layout to ensure easy access to components such as motor output terminals, board power and ground terminals, and headers designed for specific purposes such as LCD board operation or serial communication. Using the EAGLE program, most component connections were successfully created using the software generation tool. Adjustments were made to specific connection paths such as 12V supply paths between the board power terminal and the motor control terminals, where the width of the copper path between terminals was increased to improve board operation. Adjustments in the width of paths across the PCB lead to interference or overlap between various circuit connections that were otherwise unaffected. This direct change led to the requirement that a number of internal board connections be redirected during the layout design phase to avoid an internal short-circuit in the final board design. The EAGLE software made this adjustment simple, with easily-implemented drag-and-drop changes to affected paths resolving the issue with little trouble. Once all component paths were confirmed, the PCB board layout schematic, depicted in figure 1, was uploaded to an external fabrication company's website for prompt and professional service in printing the circuit board. In the case of this project, the OshPark fabrication company was used. Figure 2 depicts the PCB prior to component installation.

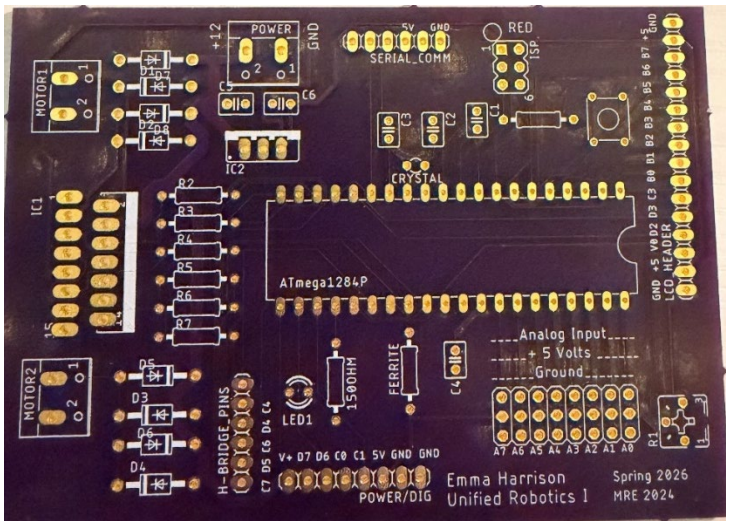


Figure 2. PCB Prior to Component Installation

Following the receipt of the fabricated boards, component installation took place in-lab. All components listed in table 1 were installed by-hand using standard soldering techniques. To prevent damaging the ATmega1284 microchip, a 40-pin socket was installed in place of the microchip, allowing for the microchip to be plugged into the board after soldering was completed. This reduced the risk of overheating and damaging the microchip during board construction substantially. For all other sensitive components, techniques of heat-sinking were employed, where a metallic clip was attached to the leg of the component which was being soldered. This metallic clip prevented excess heat transfer between the hot-end of the soldering iron and the chip's internal components. Due to lack of experience in soldering, students experienced several issues during board construction. On multiple occasions, a bead of solder would bridge two adjacent solder pads, creating a direct short circuit between two pins that would otherwise be independent. To resolve this, braided copper wire was heated to functionally remove the cooled solder from the undesired position on the board. Also, it was discovered that excessive heat application near a solder tab caused nearby plastic coatings to begin to melt or deform. Special care was taken from that point forward to ensure PCB board melting was reduced to a minimum. Figure 3 depicts the completed PCB used in this project implementation.

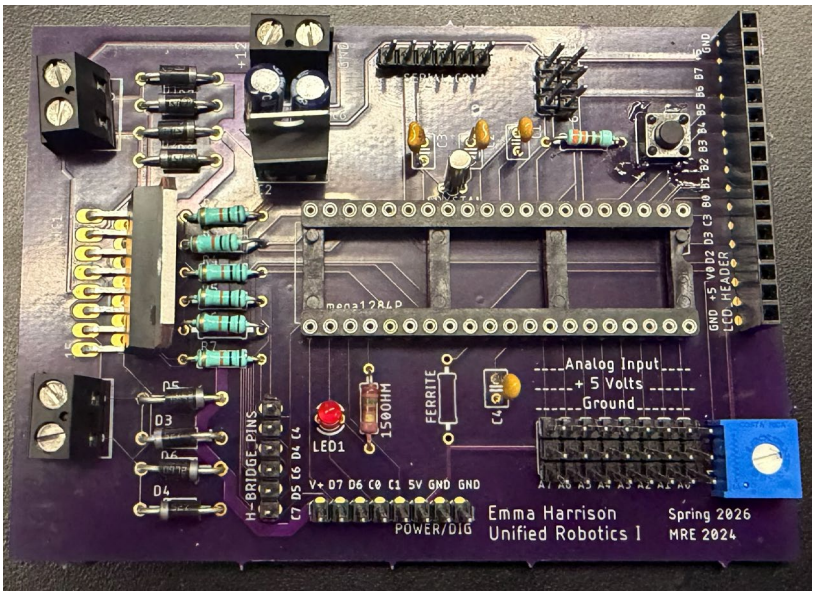


Figure 3. PCB with Components Installed

Following the completion of this board, the components were tested for continuity of vital internal board connections. Using a digital multimeter, the PCB circuit schematic was reviewed and compared to the board implementation. It was determined all relevant connections were successfully made. Following this confirmation, previously completed laboratory experiments making use of a similar PCB were reperformed to confirm PCB performance. This board was tested for direct motor control, LCD screen operation, standard logic implementation through a simple LED blinker circuit, and other standard tests. The board was not capable of displaying characters on the LCD screen, but was able to provide sufficient voltage to both motor control terminals to operate a DC motor.

Finally, in implementing this project, students were provided the chassis needed to operate the robot as intended. A simple two-tier robot frame was provided, with two DC motors attached to the front of the frame, and a free-spinning caster wheel attached to the rear for stability. Each robot also included a 12V 7Ah lead-acid battery to supply power to the microcontroller and the motors. The upper surface provided support to the laptop being used to implement the software needed for this project. Successful construction and verification of board performance marked the completion of the hardware implementation portion of this project.

SOFTWARE IMPLEMENTATION

Continuing with implementation details, students also made use of computer vision techniques to identify lane lines and perform frame processing as necessary. In this project, there is a requirement of using a camera to collect environmental information regarding the robot position. Using the integrated webcam on the LTU-supplied laptop, live image data was collected

regarding the local environment. Using Windows Forms in the .NET framework, a simple form was created that would display image data collected from the webcam. The relevant source code described in this section of the project report can be found in appendices B and C. Each frame captured was then processed using the EMGU CV library, which provides access to image processing techniques. In the case of this project, simple binary and HSV filtering was implemented to isolate pixels within a desired range of color values within the frames collected from the webcam. As the project proceeded, the processing method evolved from utilizing the white outer lane lines, to using the yellow center lane marker to simplify lane-follower logic.

To begin, each iteration of the program started with the capture of a single frame from the camera input. This frame was collected as a Mat object, allowing for simplified processing of various information. A conversion of Mat object color space type, from BGR (blue, green, red) to HSV (hue, saturation, value) was completed using a provided EMGU CV library function for color processing.

To continue with HSV filtering implementation, the threshold used to detect the yellow line within the robot's vision was tuned using trial and error of the webcam input. Given the standard lighting adjustments made automatically by the integrated webcam, this proved to be difficult. After many iterations of trial-and-error, the HSV range of values corresponding to the yellow lane lines established in the on-campus lab were determined to be as follows when implemented in EMGU CV:

EMGU Lower HSV limit: Hue = 24, Saturation = 43, Value = 255

EMGU Upper HSV limit: Hue = 30, Saturation = 255, Value = 255

Considering the limited range of values available in EMGU CV, these HSV values can be converted to actual values represented on a standard HSV chart. Considering the maximum value of a Hue is 360, Saturation is 100, and value is 100, this conversion can be completed using a simple calculation, described as follows:

$$Actual\ HSV\ value = \frac{EMGU\ HSV\ value}{EMGU\ HSV\ maximum} \cdot Actual\ HSV\ maximum$$

Using this conversion, the actual range of HSV values can be described as:

Actual Lower HSV limit: Hue = 48.3, Saturation = 16.86, Value = 100

Actual Upper HSV limit: Hue = 60.335, Saturation = 100, Value = 100

These values were obtained considering the EMGU HSV range of Hue up to 179, Saturation up to 255, and value up to 255. The colors represented by these values can be seen in figures 4a and 4b.

(a) (b)

Figure 4. (a) Lower Yellow HSV Limit and (b) Upper Yellow HSV Limit

Considering the state of the lane markings found in the laboratory, and the success of the system following this implementation, this range of HSV values was determined to be sufficient for the purposes of this project. Using a provided functionality within the EMGU CV library, any pixel found within an image object that is in the previously defined range of yellow HSV values was marked as a 1 value, or white, within a binary bitmap, and all other pixels were marked as 0, or black.

This process of trial-and-error and color tuning was repeated given the shade of the red line used for stopping and starting logic implementation within the laboratory setting. The final values used within the robotic demonstration for red HSV filtering can be viewed as follows:

EMGU Lower HSV limit: Hue = 162, Saturation = 76, Value = 102

EMGU Upper HSV limit: Hue = 179, Saturation = 178, Value = 166

These values, after performing the previously described conversion calculations, correspond to the following actual HSV values:

Actual Lower HSV limit: Hue = 325.81, Saturation = 29.8, Value = 40

Actual Upper HSV limit: Hue = 360, Saturation = 69.8, Value = 65.1

The colors represented by these values can be seen in figures 5a and 5b.



(a) (b)

Figure 5. (a) Lower Red HSV Limit and (b) Upper Red HSV Limit

Though this is an oddly-appearing range of colors, these values proved to mask the red stop and start point in the laboratory with little struggle. Figure 6 depicts the Windows Form output as debugging was taking place to tune the red HSV values.

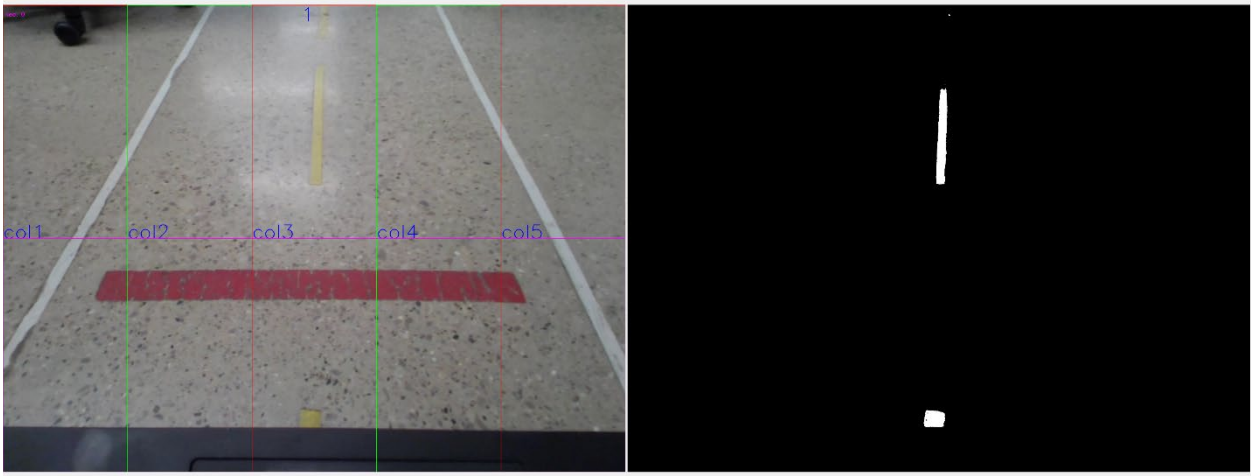


Figure 6. HSV Filter Tuning Process Image

As depicted in figure 6, the windows form display depicts not only the live camera input, but also the current bitmap corresponding to the HSV-filtered image. Output of this type allowed for direct debugging and state-machine implementation to take place, as line detection was easily and directly displayed.

Computation and decision-making processes all took place directly within the computer-vision program. The logic implementation description can be found in the state machine section of this laboratory report. As decisions were made using information collected from the visual data, such as where the yellow lane marking was located horizontally within the image frame, simple character values were sent to the ATmega1284 microcontroller using serial communication. The serial communication port number was determined through simple evaluation of the Windows Device Manager window. In the case of this project, the ATmega1284 board, when connected using the Polulu AVR programmer in a USB port, was frequently assigned to COM port 20. When the code is first initialized, the computer vision program opens this COM port and begins passing single characters to the microcontroller. The microcontroller program collects these characters by implementing functionalities provided in the course's library and header files. The values passed over serial communication correspond to actions that can be taken by the microcontroller, such as the value '1' indicating the decision has been made the robot should move forward, and send equivalent PWM values to each motor on the robot.

STATE MACHINE DESIGN & IMPLEMENTATION

In determining the logical decision-making requirement of this project, a state machine was designed and later implemented to determine the possible states of the robot, and the necessary state transitions that must take place to reach each desired state. Figure 7 provides the state transition diagram used to describe the robot's behaviors and appropriate responses.

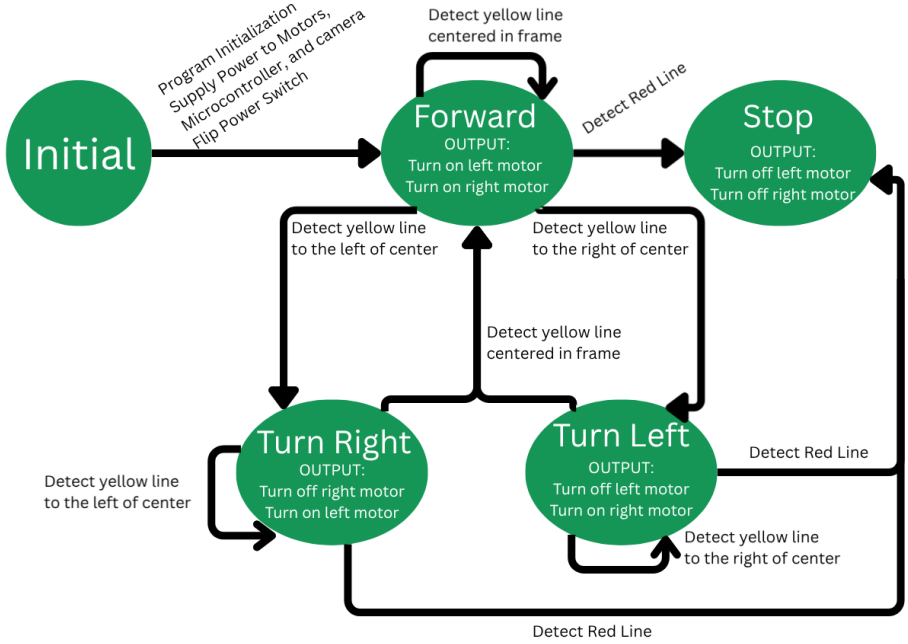


Figure 7. State Transition Diagram of Robotic State Machine

In this diagram, each possible state within the project parameters is depicted as a green oval. Each transition input is written in black ink, and outputs for each state are provided within the state symbol. At the start of the state machine, the robot must initialize. Programs are started, microcontrollers are connected, and proper preparations are made. Following this, the only input available is user input to begin the robot, such as code compilation or power supply to the necessary components. Once the necessary input is received, the robot begins moving forward. This is thanks to the known start condition of the robot on the provided track, in which the robot will always begin in the same position, on a straight portion of the lane track. Once the forward state is reached, lane detection logic begins. Code written in C# and implemented in Microsoft Visual Studio is used to detect the lane lines through image capturing and HSV filtering, as described in the previous software implementation section of this report. Once the robot is moving forward, it is always searching for and evaluating the position of the yellow lane marker within the camera frame. If the yellow line is detected in the section of the frame to the left of the center column, the robot moves to the 'Turn Right' state. To facilitate a sharp turn, the right motor is turned off completely, and the left motor is turned on at low rate, ranging between 35 and 65 given the horizontal position of the line. If the yellow line is detected at the far-left side of

the frame, the left motor receives a higher value to implement a sharper turn. Once the yellow line is returned to an acceptable centered position, the robot returns to the ‘Forward’ state. This returns the robot to equal movement of both the left and right motors. Similarly, if the yellow line is detected using HSV filtering to be on the right side of the image frame, the robot moves to the ‘Turn Left’ state, as depicted in figure 7. This state is facilitated by reducing the PWM supply value to the left motor to 0, and supplying a value between 35 and 65 to the right motor, dependent again on the horizontal position of the line within the image frame. Again, in this state, if the yellow line is detected to return to a centered position, the robot resumes standard forward motion. Finally, all motion-related states are provided a transition to the ‘Stop’ state, as well as a transition to a repetition of the current state. The stop-state transition is implemented using the red-line HSV filtering technique described in the previous section. The repeated state transition is also implemented using HSV filtering, again making a determination regarding the robot’s motion based on the position of the yellow pixels within the image frame. When a sufficient number of pixels falling within the tuned red HSV range are detected within the frame, the ‘Stop’ state is triggered, reducing the PWM value supplied to both the left and right motors to 0.

To continue, this robot state transition logic is clearly implemented within the code developed for this project. Using a series of if-else statements, alongside the described frame and image processing code, the state transitions were easily implemented. The following code snippet provides the conditional logic implemented in this project:

```
if (red > 100000)
{
   flag = true;
   decision = 0;
}
else
{
   if (yellowPixels < 500)
   {   }
   else if (ypcol3 > ypcol4 && ypcol3 > ypcol2)
   {
      decision = 1; //FORWARD
   }
   else if (ypcol4 > 150)
   {
      decision = 3; //turn a bit right
   }
   else if (ypcol2 > 150)
   {
      decision = 2; //turn a bit left
   }
   else if (ypcol1 > 150)
```

```
{
   decision = 5; //turn hard left
}
else if (ypcol5 > 150)
{
   decision = 4; //turn hard right
}
```

Once the robot has been moved past the initial state by user input, the forward state is initialized. Before deciding on a direction to move, the robot ensures the transition to the stop state, triggered by a detection of the red line, is invalid. This is implemented by testing the number of red pixels within the frame, stored in the ‘red’ variable. If the transition from forward to stop is invalid, the code then checks the transitions from the ‘Forward’ state to either the ‘Turn Right’ or ‘Turn Left’ states. If the yellow line is farther to the right or left of the center of the image, the robot transitions to the appropriate state within the code using the count of yellow pixels in each column. For reference, the image frame of the robot is divided into 5 equal-width columns. When the robot is centered along the yellow line, the center column, or column 3, contains the highest number of yellow pixels. When the robot is skewed to the left of the yellow line, or performing a left turn, the number of yellow pixels in column 2, or the second column from the right, is the highest. When the robot is skewed to the right of the yellow line, or performing a right turn, the number of yellow pixels in column 4, or the fourth column from the right, is the highest. This value-checking logic implements the state machine transitions, as each state transition tests the current state of yellow-line detection before deciding which state transition is valid.

Finally, in implementing this logic, the computational load was distributed between two processors to optimize system performance. The computer vision, directional logic, frame display and conversion of frames between images and binary bitmaps was completed using the provided laptop, equipped with a high-level processor and built-in webcam for simplicity. This code can be found in appendix B. The actual motor control logic, in which the PWM value provided to each motor is determined, was implemented on the lower-level ATmega1284 microcontroller. This distribution placed the majority of the computational burden on the higher-speed laptop processor, and prevented overloading the microcontroller. This allowed for as-fast-as-possible motor control response, as well as efficient communication between the processors using serial communication ports.

TESTING AND DEBUGGING

In completing a project of this type, issues are bound to arise, and debugging techniques are necessary to overcome difficult obstacles. Throughout the implementation process, many issues presented themselves, regarding hardware, software, and communication techniques.

To begin, as mentioned previously, issues were encountered with PCB design and construction. During the PCB design process, mistakes were easily made regarding component connections within the circuit diagram. On multiple occasions, a node was accidentally created between two wires whose leads overlapped within the schematic. These nodes were manually deleted within EAGLE cad, to prevent unintentional component connections. The PCB circuit schematic was compared directly against an example schematic alongside component data sheets to test that all proper connections were made, and all improper connections were removed. Also, when constructing the PCB, continuity tests were also performed using a digital multimeter to determine if needed connections were present within the board. This provided vital insight into the board connections, existing circuits, and connections that were to be completed by a soldered-in component.

To continue, in developing the computer-vision program for this project, many tests and debugging steps took place. For example, in the early phases of the project, the camera vision code was only required to display the camera input and related binary bitmask. This simple implementation alone required multiple hours of determining proper libraries to install, refining coding practices, and developing an in-depth understanding of the Windows Forms platform. In further development, the binary bitmask was tuned to view the lane markings to be followed by the robot. This included trial-and-error testing, in which the bitmask's sensitivity would be slightly adjusted. To complete this testing, the robot was placed on the floor within the lane, and only the camera vision code was launched. This display was noted during each test iteration, and additional changes were made. A series of erosions and dilations were performed on the white pixels detected within the mask, after many iterations of testing, to remove random noise and increase the weight of actual white pixels within the mask. Through testing this mask multiple times, an optimal map depicting white lane lines only was obtained. Random noise was discovered to result from an automatically-adjusting camera, alongside the composition of the floor surrounding the robot including light-colored aggregate that, under bright laboratory lights, fell within the binary threshold values.

Further, in implementing the motor controller, there were tests to be performed, both regarding the PCB construction and the motor control logic. During many attempts to operate both motors simultaneously, one motor remained non-operational. Tests were performed using a digital multimeter to determine where the power supply error or logical mistake was occurring within the board. Using the PCB circuit schematic, the connections between the ATmega1284 and the L298N dual H-bridge were determined and tested for DC voltage. This includes the PWM enable pins, directional control pins, and H-bridge output pins. During this inspection, it was determined

that within the non-operational motor, a high value for both the forward and reverse directional pins was being provided by the ATmega1284 microcontroller, resulting in zero voltage across the motor itself. Following review of the ATmega1284 microcontroller component datasheet [1], it was determined that the PC4 and PC5 pins are classified as JTAG-reserved pins, and must be reset in order to perform general-input-output actions such as direct motor control [2]. Provided this information, the pins were reset using the following command within the Microchip Studio code:

```
MCUCR |= (1 << JTD);
```

```
MCUCR |= (1 << JTD);
```

This command was provided to disable the JTAG (Joint Test Action Group) pins. At the initialization of the microchip studio code, these commands are performed. Following this update, both motors operated as intended.

To continue with testing and debugging, issues were encountered during the serial communication process. Given that the board constructed for the project was not able to operate an LCD properly, the previously established method of testing serial communication through LCD display was no longer available. Once serial communication was established within the C# Windows Forms code, issues arose when receiving the designated characters. To debug the issues, the laboratory experiment implementing serial communication was returned to. It was discovered that, since this was a new microchip, the CKDIV8 fuse was still active within the chip, reducing the clock frequency to 1k Hz. This was adjusted, though it was again discovered serial communication was unsuccessful. Continuing with the experiment instructions, the issue was discovered in the baud rate being used by both the Windows Forms application and the microcontroller. The microcontroller implements an 11.059 MHz crystal, as mentioned within the serial communication library provided for the course, and thus must implement a corresponding CPU frequency and a 9600 baud rate. Once this correction was made, serial communication was completed successfully.

Also, once all components were operational, there were clear bugs in the movement logic implemented on the robot within the Windows Forms application. Initially, white-lane following was implemented, and the objective provided was to follow one white line and maintain a set distance to the right of the line. This was ineffective, as the bitmask was not partitioned to only evaluate the right side of the frame input, and thus would attempt to stay to the right of both white segments simultaneously. This also provided issue when the robot would encounter a ‘bottleneck’ or narrowed section of the track, in which following to a set distance away from the white line would cause the robot to exit the track. After encountering these issues, the decision was made to transition to yellow-line following. In initial phases, following correct HSV tuning, the robot would startup, and follow straight lines well, but turn in the wrong direction at any curve in the path. It was quickly determined that the line-following logic was reverse of the

image presented within the Windows Forms display. Any designation that the lane curved to the left would lead to a sharp right turn, and the robot would lose sight of the yellow line completely. To resolve this, line-following logic was simply switched within the ATmega1284 microcontroller code, where the value sent to the right motor was assigned to the left motor, and the value originally sent to the left motor was then sent to the right. Upon further testing along road curves and challenging track areas, the robot performed with much more agreeability.

Finally, a simple issue encountered was related to the range of vision of the robot. Within a particular portion of the track, there was a plastic storage shelf whose bottom platform was visible within the robot frame. Coincidentally, part of the platform is a shade of yellow that falls within the HSV mask threshold. This section of the shelf would skew the robot's motion, as it would attempt to center the shelf's yellow section within the frame. This would lead to unexpected motion and exiting of the lane lines. To resolve this, there were attempts to further tune the HSV mask to exclude that shade of yellow. Upon further examination of the shelf, it was determined that the yellow portion of the shelf was only visible to the robot if the robot passed the shelf on the right. By reversing the direction in which the robot moves around the track, the troublesome yellow section was avoided, and the issue was resolved.

RESULTS, DISCUSSION & NEXT STEPS

Of course, in completing this project, there are notable results to be presented. These results can be determined as follows:

- The microcontroller and related code were able to successfully operate the motors as desired
- The ATmega1284 was successfully able to receive commands from the Windows Forms application code over serial communication using the COM20 port
- The Windows Forms application code was successfully able to obtain live camera input from the integrated webcam, perform relevant HSV masking to detect particular shades of both yellow and red lane markings, and implement state machine logic to choose the robot action based on the immediate input

To expand on these points, the microcontroller was successfully designed, ordered, fabricated and tested throughout the course of the semester. Rigorous testing and debugging led to the determination that the microcontroller was usable within the parameters of this assignment. Performing a pin-level analysis of various errors resulted in the direct improvement of the board performance, and led to the ability for direct motor control to be completed. Communication between the Windows Forms program and the microcontroller was implemented using serial communication and the COM20 port after, again, rigorous testing and debugging processes. After discovering a discrepancy between the board's actual baud rate and clock frequency and the expected baud rate and clock frequency, the issue was resolved and communication continued as expected. Through deliberate coding and multiple iterations, the Windows Forms

application code was able to obtain live information from the integrated webcam. HSV filtering techniques were tested through trial-and-error to ultimately determine the ideal range of values to isolate each desired input. State machine logic was implemented through a simple if-else conditional testing structure, allowing each state transition to be tested within each code iteration in order of importance to determine which state is the most appropriate for the current input. These successes were derived from many hours in-lab testing, debugging, and discussing with peers and developing working theories and eventual plans on how to resolve such issues. These results clearly address the project goals of implementing a state machine, and designing a robot that can follow a lane placed upon the floor of a room without exiting the white lane lines, and both stopping and starting near the red lane marking on the floor.

During the demonstration of the robot and preliminary testing, it was noticed that the robot oscillates around the yellow lane marking line, rather than following it directly straight. This is due to the logic implementation only using 5 columns across the frame, resulting in a crude lane-following behavior. While the robot never exited the desired path, during straight segments in which the robot could have moved faster and required less computation, the robot instead would drift to one side, correct harshly, then drift to the other side of the line. This process continued as the robot moved throughout the course, always drifting side-to-side rather than directly following the lane in a straight line. In the future to resolve this, the number of columns used can be increased, to further refine the turning logic and prevent overturning when only a small adjustment is necessary. Also, other techniques within the EMGU CV library structure can be implemented, such as shape detection. In a future implementation, the rectangular shape of a yellow lane marking can be detected, and the orientation of that rectangle can be used to inform on the robot's position and necessary next step. If the centroid of the detected rectangle is skewed from the center of the frame, the robot can perform a proportional adjustment to recenter the rectangle.

Further improvements can be made to the robot, to increase task completion efficiency and reduce the capability for error. For example, if the robot was able to detect a straight-line path ahead, the robot could increase overall speed to complete the course faster, given that the straight-line path requires less in-depth decision-making. The robot could also reduce the number of computations completed within these regions, as a straight-line path reduces the need for decision-making. This would reduce the computational load of the robot at any given time. Also, to reduce errors in HSV masking, other techniques for lane following can be employed, such as the use of an AI model, such as YOLO (You Only Look Once), to identify lane lines and markings, and develop a behavior-based approach to solving the lane-following problem. Also, if the robot were to be equipped with additional sensors, such as lidar or an ultrasonic distance sensor, obstacle avoidance behaviors could be implemented to reduce collisions. During testing, there were instances when the robot was off-course, and suffered a near-miss with nearby tables, chairs or even students as the robot failed to properly navigate. The addition of obstacle avoidance behaviors could vastly reduce the chance of collision, especially during testing phases.

SELF-REFLECTION

In working through a complex problem such as this assignment, I have gained many new skills in problem-solving, programming, and circuitry in general. In designing, ordering and constructing a PCB, I strengthened my understanding of basic electronics and electronic components while also gaining a new understanding of microcontrollers and IC chips. I gained functional skills in soldering and component identification and testing, as I constructed the board based on the developed circuit schematic. I also gained many skills in programming, both in the C and C# programming languages, as well as implementing Windows Forms application programs. I have developed a better understanding of the concepts behind computer vision, including HSV and binary filtering, as well as the use of Mat and Image objects to process and convert image frames as needed to complete complex computations. I have gained experience in debugging a serial communication system, including the identification of relevant fuse settings, baud rate and clock frequency values that can cause serial communication to fail. I have gained real-world skills in reviewing component data sheets, and making use of industrial-standard component documentation to debug and determine issues present in a system. I have also gained experience using supplemental libraries within the Microchip Studio application, which greatly increases the programming capacity I can implement, while simultaneously reducing the complexity of any particular code implementation.

In working through complex issues, I have also, of course, identified points of weakness that can be improved upon. I can improve my problem-solving and debugging skills to reduce the amount of time I spend on any singular issue. I need to further refine my ability to determine the root cause of a particular problem before attempting to change any variable to improve the issue. I must develop more patience in debugging, as I have noticed I tend to try to change variables before understanding the problem completely, and can worsen the issue unintentionally. Further, I need to gain a more-comprehensive understanding of electronics, as well as the purpose of each component implemented upon a PCB.

The lessons gained from completing this project can certainly be applied to future projects, as these vital techniques in programming and PCB design and construction make up the foundation of many more-complex processes in robotics engineering. For example, I can employ my knowledge of C programming within the Microchip Studio platform with other microcontroller applications, and make use of provided libraries to simplify code. I can also implement the Windows Forms application toward other projects that require a user interface. I can apply my new skills in PCB design and construction toward other projects that require a custom PCB and particular components. I can also apply my state machine from this project to future projects related to lane-following logic, including the Intelligent Ground Vehicle Competition.

CONCLUSIONS

To conclude, this project addresses the assignment to program and implement a robot that is capable of detecting and following lane markings without deviating from the provided path, using computer vision and implementing a dual H-bridge motor controller using a custom-

designed printed circuit board and an ATmega1284 microcontroller. The results of the completion of this assignment include the successful implementation of code in the C programming language to implement dual motor control, serial communication between programming languages and platforms to distribute computational load between two processors of varying speed and capabilities, and code to obtain live camera input from an integrated webcam to inform on the robot's position and make use of a state machine to determine the best response to the current input. This system can be improved by reducing the oscillation of the robot about the yellow lane markings by increasing the number of columns across the width of the image frame being evaluated, and reducing the intensity of each motor response when applicable. Also, future applications could consider alternative lane-detection techniques and the addition of obstacle avoidance to the system framework. Completing this assignment has granted students new skills in problem solving, computation and programming, computer vision, and many other areas vital to a successful career in robotics engineering.

REFERENCES

[1] "ATmega164A/PA/324A/PA/644A/PA/1284/P megaAVR Data Sheet," Microchip, [Online]. Available: https://ww1.microchip.com/downloads/aemDocuments/documents/MCU08/ProductDocuments/DataSheets/ATmega164A_PA-324A_PA-644A_PA-1284_P_Data-Sheet-40002070B.pdf. [Accessed 30 January 2026].

[2] "Standalone ATmega1284P," Riot, [Online]. Available: https://guide.riot-os.org/boards/atmega1284p/. [Accessed 8 May 2026].

APPENDICES

Appendix A

PCB Schematic Design

This appendix contains the PCB circuit schematic design used to implement and order a custom circuit board implemented in this assignment.

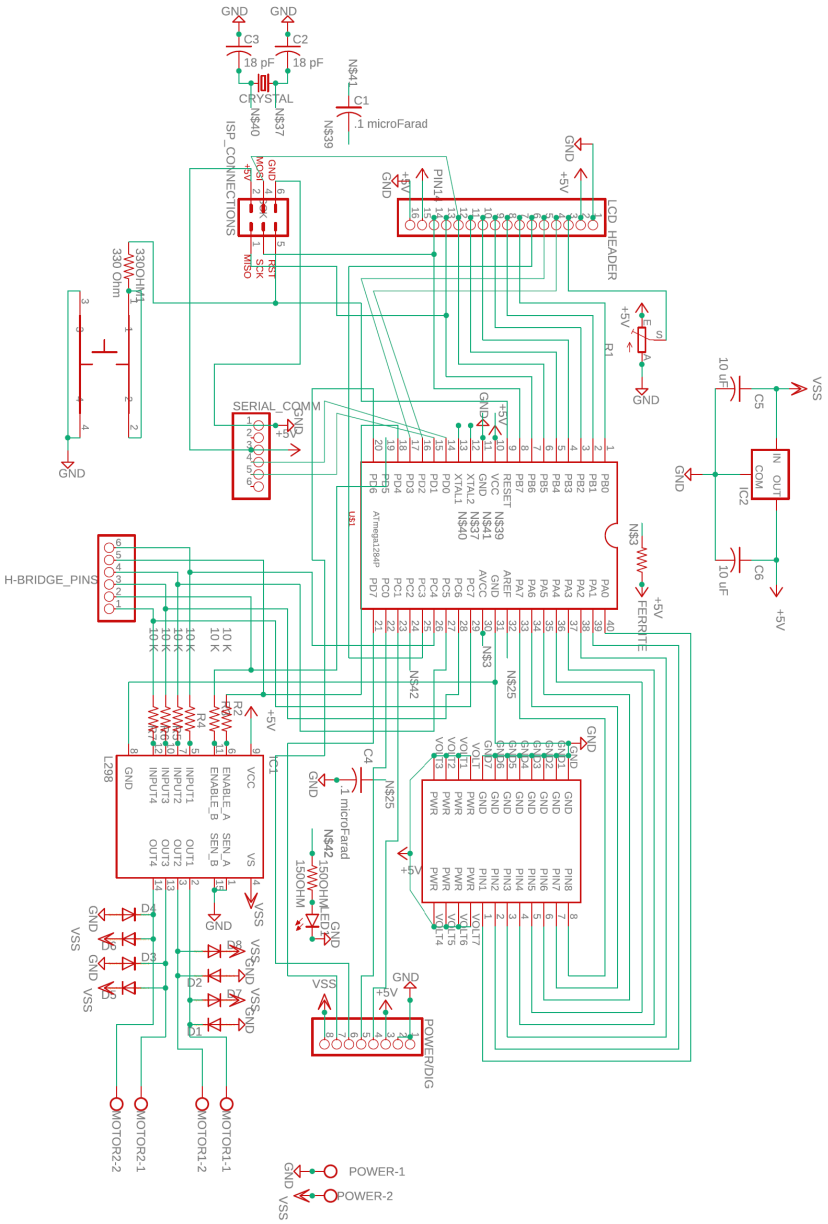


Figure 8. PCB Schematic Diagram

Appendix B

Source Code for Computer Vision Implementation

This appendix contains the source code implemented within the Windows Forms application to perform computer vision computations and implement the designed state machine.

```
using Emgu.CV;
using System;
using System.Windows.Forms;
using Emgu.CV.Structure;
using System.Drawing;
using System.Threading;
using System.Collections.Specialized;
using Emgu.CV.CvEnum;
using Emgu.CV.Util;
using System.IO.Ports;
using System.Threading.Tasks;


namespace CS_HW_2_Emma_Harrison
{
   public partial class Form1 : Form
   {
      VideoCapture _capture;
      Thread _captureThread;
      SerialPort _serialPort;// = new SerialPort("COM4", 9600);

      public Form1()
      {
         InitializeComponent();

      }

      private void Form1_Load_1(object sender, EventArgs e)
      {
         //create the capture object and processing thread
         _capture = new VideoCapture(0); //0 -> device webcam
         _captureThread = new Thread(DisplayWebcam);
         _serialPort = new SerialPort("COM20", 9600); //change com port as needed for ATmega
Serial Port (determined using Device Manager)
         _serialPort.DataBits = 8;
```

```
    _serialPort.Parity = Parity.None;
    _serialPort.StopBits = StopBits.One;
    _serialPort.Open();

    _captureThread.Start();
}

private void DisplayWebcam()
{
    int red_start = 0;
    bool already_moving = false;

    while (_capture.IsOpened)
    {
        //frame maintenance
        Mat frame = _capture.QueryFrame();

        //initialize variables for value counting

        //white pixel counting
        int wpcol1 = 0; //far left
        int wpcol2 = 0; //second from left
        int wpmid = 0; //third from left
        int wpcol4 = 0; //middle
        int wpcol5 = 0; //third from right
        int wpcol6 = 0; //second from right
        int wpcol7 = 0; //far right

        //count red pixels
        int red = 0;

        //count yellow pixels
        int yellowPixels = 0;
        int yellowPixelsMiddle = 0;

        int ypcol1 = 0;
        int ypcol2 = 0;
        int ypcol3 = 0;
        int ypcol4 = 0;
        int ypcol5 = 0;

        //flag for red stop logic
        bool flag = false;
```

```
            //resize to PictureBox aspect ratio
            int newHeight = (frame.Size.Height * pictureBox1.Size.Width) / frame.Size.Width;
            Size newSize = new Size(pictureBox1.Size.Width, newHeight);
            CvInvoke.Resize(frame, frame, newSize);

            //grayscaling and binary thresholding
            Mat grayscale = new Mat();
            CvInvoke.CvtColor(frame, grayscale,
Emgu.CV.CvEnum.ColorConversion.Bgr2Gray);
            var mean = CvInvoke.Mean(grayscale);

            Mat binary_thresh = new Mat();
            CvInvoke.Threshold(grayscale, binary_thresh, 175, 255,
Emgu.CV.CvEnum.ThresholdType.Binary);

            //hsv thresholding
            //red hsv mat object
            Mat hsv = new Mat();
            CvInvoke.CvtColor(frame, hsv, Emgu.CV.CvEnum.ColorConversion.Bgr2Hsv);
            Image<Hsv, byte> redhsvImage = hsv.ToImage<Hsv, byte>();
            //175,255

            //yellow hsv mat object
            Mat hsvYellow = new Mat();
            CvInvoke.CvtColor(frame, hsvYellow,
Emgu.CV.CvEnum.ColorConversion.Bgr2Hsv);
            Image<Hsv, byte> yellowHSVimg = hsvYellow.ToImage<Hsv, byte>();

            //initialize images for binary thresholding columns
            Image<Gray, byte> img = binary_thresh.ToImage<Gray, byte>();
            Image<Gray, byte> img2 = binary_thresh.ToImage<Gray, byte>();
            Image<Gray, byte> col1 = binary_thresh.ToImage<Gray, byte>();
            Image<Gray, byte> col2 = binary_thresh.ToImage<Gray, byte>();
            Image<Gray, byte> col3 = binary_thresh.ToImage<Gray, byte>();
            Image<Gray, byte> col4 = binary_thresh.ToImage<Gray, byte>();
            Image<Gray, byte> col5 = binary_thresh.ToImage<Gray, byte>();
            Image<Gray, byte> col6 = binary_thresh.ToImage<Gray, byte>();
            Image<Gray, byte> col7 = binary_thresh.ToImage<Gray, byte>();

            //erode and dilate to reduce noise and reinforce lane lines
            img = img.Erode(3).Dilate(1);
```

```
            img2 = img2.Erode(3).Dilate(1);
            col1 = col1.Erode(3).Dilate(1);
            col2 = col2.Erode(3).Dilate(1);

            col3 = col3.Erode(3).Dilate(1);
            col4 = col4.Erode(3).Dilate(1);
            col5 = col5.Erode(3).Dilate(1);
            col6 = col6.Erode(3).Dilate(1);
            col7 = col7.Erode(3).Dilate(1);

            //count the white pixels in each column
            wpcol1 = img.CountNonzero()[0];
            wpcol2 = img2.CountNonzero()[0];
            wpmid = col3.CountNonzero()[0];
            wpcol4 = col4.CountNonzero()[0];
            wpcol5 = col5.CountNonzero()[0];
            wpcol6 = col6.CountNonzero()[0];
            wpcol7 = col7.CountNonzero()[0];

            //tune red HSV filter
            Hsv redlowerLimit = new Hsv(162, 76, 102);//325, 30, 40);
            Hsv redupperLimit = new Hsv(179, 178, 166);//360, 70, 65);
            //345.5, 51, 47

            //establish region of interest and pixel couting for red HSV mask
            redhsvImage.ROI = new Rectangle(0, 0, frame.Width, frame.Height);
            Image<Gray, byte> redmask = redhsvImage.InRange(redlowerLimit, redupperLimit);
            redhsvImage.ROI = new Rectangle(0, frame.Height - frame.Height / 2, frame.Width,
frame.Height);
            Image<Gray, byte> countRed = redhsvImage.InRange(new Hsv(0, 150, 100), new
Hsv(25, 255, 255));
            red = redmask.CountNonzero()[0];
            CvInvoke.PutText(frame, $"Red: {red}", new Point(10, 70),
FontFace.HersheySimplex, 1.2, new MCvScalar(255, 0, 255), 2);

            //tune yellow HSV filter
            Hsv lowerLimit = new Hsv(24, 43, 122);
            Hsv upperLimit = new Hsv(30, 255, 255);

            //establish yellow hsv masks
            Image<Gray, byte> mask = yellowHSVimg.InRange(lowerLimit, upperLimit);
            yellowPixelsMiddle = mask.CountNonzero()[0];
```

```
            Image<Gray, byte> ycol1 = yellowHSVimg.InRange(lowerLimit, upperLimit);
            Image<Gray, byte> ycol2 = yellowHSVimg.InRange(lowerLimit, upperLimit);
            Image<Gray, byte> ycol3 = yellowHSVimg.InRange(lowerLimit, upperLimit);
            Image<Gray, byte> ycol4 = yellowHSVimg.InRange(lowerLimit, upperLimit);
            Image<Gray, byte> ycol5 = yellowHSVimg.InRange(lowerLimit, upperLimit);

            //establish yellow ROIs for pixel counting
            ycol1.ROI = new Rectangle(0, frame.Height - frame.Height / 2, frame.Width / 5,
frame.Height);
            ycol2.ROI = new Rectangle((frame.Width) / 5, frame.Height - frame.Height / 2,
frame.Width / 5, frame.Height);
            ycol3.ROI = new Rectangle(2 * (frame.Width) / 5, frame.Height - frame.Height / 2,
frame.Width / 5, frame.Height);
            ycol4.ROI = new Rectangle(3 * (frame.Width) / 5, frame.Height - frame.Height / 2,
frame.Width / 5, frame.Height);
            ycol5.ROI = new Rectangle(4 * (frame.Width) / 5, frame.Height - frame.Height / 2,
frame.Width / 5, frame.Height);

            //count yellow pixels in whole image
            yellowHSVimg.ROI = new Rectangle(0, 0, frame.Width, frame.Height);
            yellowPixels = yellowHSVimg.CountNonzero()[0];

            //count yellow pixels in each ROI
            ypcol1 = ycol1.CountNonzero()[0];
            ypcol2 = ycol2.CountNonzero()[0];
            ypcol3 = ycol3.CountNonzero()[0];
            ypcol4 = ycol4.CountNonzero()[0];
            ypcol5 = ycol5.CountNonzero()[0];

            //draw yellow ROIs on camera input
            CvInvoke.Rectangle(frame, ycol1.ROI, new MCvScalar(0, 0, 255), 2);
            CvInvoke.Rectangle(frame, ycol2.ROI, new MCvScalar(0, 255, 0), 2);
            CvInvoke.Rectangle(frame, ycol3.ROI, new MCvScalar(0, 0, 255), 2);
            CvInvoke.Rectangle(frame, ycol4.ROI, new MCvScalar(0, 255, 0), 2);
            CvInvoke.Rectangle(frame, ycol5.ROI, new MCvScalar(0, 0, 255), 2);

            //label each column for easy debugging
            CvInvoke.PutText(frame, "col1", new Point(0, frame.Height - frame.Height / 2),
FontFace.HersheySimplex, 4, new MCvScalar(255, 0, 0), 3);
            CvInvoke.PutText(frame, "col2", new Point((frame.Width) / 5, frame.Height -
frame.Height / 2), FontFace.HersheySimplex, 4, new MCvScalar(255, 0, 0), 3);
            CvInvoke.PutText(frame, "col3", new Point(2 * (frame.Width) / 5, frame.Height -
frame.Height / 2), FontFace.HersheySimplex, 4, new MCvScalar(255, 0, 0), 3);
```

```
        CvInvoke.PutText(frame, "col4", new Point(3 * (frame.Width) / 5, frame.Height -
frame.Height / 2), FontFace.HersheySimplex, 4, new MCvScalar(255, 0, 0), 3);
        CvInvoke.PutText(frame, "col5", new Point(4 * (frame.Width) / 5, frame.Height -
frame.Height / 2), FontFace.HersheySimplex, 4, new MCvScalar(255, 0, 0), 3);

        //draw ROI rectangles on white input (if using white line logic - not implemented here
        CvInvoke.Rectangle(mask, img.ROI, new MCvScalar(0, 0, 255), 2);
        CvInvoke.Rectangle(mask, img2.ROI, new MCvScalar(0, 255, 0), 2);
        CvInvoke.Rectangle(mask, col3.ROI, new MCvScalar(0, 0, 255), 2);
        CvInvoke.Rectangle(mask, col4.ROI, new MCvScalar(0, 255, 0), 2);
        CvInvoke.Rectangle(mask, col5.ROI, new MCvScalar(0, 0, 255), 2);

        //draw red mask ROI on image frame
        CvInvoke.Rectangle(frame, redhsvImage.ROI, new MCvScalar(255, 0, 255), 2);
        int decision = 0;

        if (red > 100000) //if there is a prominent red line in the image
        {
           flag = true; //mark you have seen the red line
           decision = 0; //send stop command to atmega
        }
        else
        {
           //decision
           // 0 -> STOP
           // 1 -> FORWARD
           // 2 -> TURN LEFT
           // 3 -> TURN RIGHT
           // 4 -> HARD RIGHT
           // 5 -> HARD LEFT
           // | col5 | col4 | col3 | col2 | col1 |
           if (yellowPixels < 500)
           {
              //decision = 0; //STOP! No line! //did not use logic, would cause robot to
unexpectedly stop
           }
           else if (ypcol3 > ypcol4 && ypcol3 > ypcol2) //if there are more yellow pixels in the
middle than either of the neighboring columns
           {
              decision = 1; //FORWARD
           }
           else if (ypcol4 > 150) //otherwise if there are notaable yellow pixels in the 4th
column (left of center)
```

```
                {
                    decision = 3; //turn a bit right
                }
                else if (ypcol2 > 150) //otherwise if there are notable yellow pixels in the 2nd
column (right of center)
                {
                    decision = 2; //turn a bit left
                }
                else if (ypcol1 > 150) //otherwise if there are notable yellow pixels in the first
column (far right of center)
                {
                    decision = 5; //turn hard left
                }
                else if (ypcol5 > 150) //otherwise if there are notable yellow pixels in the last
column (far left of center)
                {
                    decision = 4; //turn hard right
                }
                else
                {
                    //decision = 8; //ignore default case -> caused errors in logic
                }

            }

            if (flag) //if you have seen the red line already
            {
                decision = 0; //e-stop - must restart to continue
            }
            _serialPort.Write(decision.ToString()); //write decision string to serial port
            Thread.Sleep(50); //wait before completing next decision

            //format display of decision value
            CvInvoke.PutText(frame, decision.ToString(), new Point(frame.Width / 2 - 80, 100),
FontFace.HersheySimplex, 4, new MCvScalar(255, 0, 0), 3);

            //display the image in the PictureBox
            Bitmap bmp = frame.ToBitmap(); //frame bitmap
            Bitmap binaryBmp = binary_thresh.ToBitmap();  //binary thresholded bitmap
            Bitmap yellowMask = mask.ToBitmap(); //yellow hsv mask bitmap
            Bitmap redMask = redmask.ToBitmap();  //red hsv bitmap
            Mat combinedMask = new Mat(); //create new mat object for combined mask
```

```
            CvInvoke.BitwiseOr(mask, redmask, combinedMask); //combine yellow and red
masks into one mask for display purposes
            Bitmap comboMask = combinedMask.ToBitmap(); //create combined mask bitmap
            pictureBox2.Invoke(new Action(() =>
            {
               pictureBox2.Image = bmp; //display frame bitmap in windows form
            }));

            pictureBox3.Invoke(new Action(() =>
            {
               pictureBox3.Image = comboMask;//binaryBmp; //display combined bitmap in
windows form
            }));

            //cleanup ROIs
            img.ROI = Rectangle.Empty;
            img2.ROI = Rectangle.Empty;
            col3.ROI = Rectangle.Empty;
            col4.ROI = Rectangle.Empty;
            col5.ROI = Rectangle.Empty;

            ycol1.ROI = Rectangle.Empty;
            ycol2.ROI = Rectangle.Empty;
            ycol3.ROI = Rectangle.Empty;
            ycol4.ROI = Rectangle.Empty;
            ycol5.ROI = Rectangle.Empty;

         }
         _serialPort.Close(); //clean up serial port
      }

      private void Form1_FormClosing(object sender, FormClosingEventArgs e)
      {
         //terminate the image processing thread to avoid orphaned processes
         //_captureThread.Abort();
         if (_captureThread != null && _captureThread.IsAlive)
         {
            _captureThread.Join();
         }

         _capture?.Dispose();
      }
```

```
        private void SerialPort_DataReceived(object sender, SerialDataReceivedEventArgs e)
//handle exceptions from serial communication debugging
        {
            try
            {
                string data = _serialPort.ReadExisting();
                Console.WriteLine("MCU: " + data);
            }
            catch (Exception ex)
            {
                Console.WriteLine("Serial read error: " + ex.Message);
            }
        }

        private void pictureBox1_Click(object sender, EventArgs e)
        {

        }

        private void pictureBox3_Click(object sender, EventArgs e)
        {

        }

        private void tableLayoutPanel2_Paint(object sender, PaintEventArgs e)
        {

        }

        /*private void Form1_Load_1(object sender, EventArgs e)
        {

        }*/
    }
}
```

Appendix C

Source Code for Microcontroller Motor Control

This appendix contains the source code implemented within the Microchip Studio application to perform motor control and obtain serial communication from the Windows Forms application code.

```
/*
 * Final_Project_UR1.c
 *
 * Created: 4/25/2026 12:50:55 PM
 * Author : eharriso1
 */


#include "../../MRE_lib/F_cpu_lib.h"
#include <avr/io.h>
#include <avr/interrupt.h>
#include <stdio.h>
#include <util/delay.h>
#include "../../MRE_lib/LCD_lib.h"
#include "../../MRE_lib/A2D_lib.h"
#include "../../MRE_lib/Serial_lib2.h"
#include "../../MRE_lib/Motor_PWM_lib.h"


int main(void)
{
	MCUCR |= (1 << JTD);
	MCUCR |= (1 << JTD);
	LCD_init();
	LCDGoToPosition(1,1);
	char array[20];
	char decision;
	sprintf(array, "Hello World!!");
	LCDSendString(array);
	_delay_ms(2000);
	LCDClearScreen();
	DDRC |= 1 << PINC2;
	AD_init();
```

```
	USART_vInit();
	sei();
	HBridgeInit(); // define ports and set up 16 bit timer TCNT1
	DDRA &= ~((1<<DDA0)|(1<<DDA1));
	DDRB |= (1<<PINB7) | (1<<PINB6) | (1<<PINB5) | (1<<PINB4);
	uint8_t left = 0;
	uint8_t right = 1;
	uint8_t lspeed;
	uint8_t rspeed;
	uint8_t forward = 0;
	uint8_t reverse = 1;
	uint8_t flag = 0;

while (1)
{
		decision = USART_vReceiveByte();

		sprintf(array, "Emma Harrison");
		LCDSendString(array);

		if (!decision)
		{
			decision = 0;
		}

		 //0 -> STOP
		 //1 -> FORWARD
		 //2 -> TURN RIGHT
		 //3 -> TURN LEFT

		 //HBridgeCommand(uint8_t side, uint8_t speed, uint8_t direction);
		 // where side 0 = left motor, 1 = right
		 // speed = 0-100
		 // direction 0 = forward, 1 = reverse

		if(decision == '0') //STOP
		{
			lspeed = 0;
			rspeed = 0;
```

```
            HBridgeCommand(left,lspeed,forward);
            //_delay_ms(1000);
            HBridgeCommand(right,rspeed,forward);
            _delay_ms(10);
    }
    else if(decision == '1') //FORWARD
    {
            lspeed = 45;
            rspeed = 45;

            HBridgeCommand(left,lspeed,forward);
            //_delay_ms(1000);
            HBridgeCommand(right,rspeed,forward);
            _delay_ms(10);
    }
    else if(decision == '3') //TURN RIGHT
    {
            lspeed = 35;//65;
            rspeed = 0;//30;

            HBridgeCommand(left,lspeed,forward);
            //_delay_ms(1000);
            HBridgeCommand(right,rspeed,forward);
            _delay_ms(10);
    }
    else if(decision == '2') //TURN LEFT
    {
            lspeed = 0;//30;
            rspeed = 35;//65;

            HBridgeCommand(left,lspeed,forward);
            //_delay_ms(1000);
            HBridgeCommand(right,rspeed,forward);
            _delay_ms(10);
    }
    else if(decision == '5') //more left
    {
            lspeed = 0;//20;
            rspeed = 65;
```

```
            HBridgeCommand(left,lspeed,forward);
            //_delay_ms(1000);
            HBridgeCommand(right,rspeed,forward);
            _delay_ms(10);

    }
    else if(decision == '4') //more right
    {
            lspeed = 65;
            rspeed = 0;//20;

            HBridgeCommand(left,lspeed,forward);
            //_delay_ms(1000);
            HBridgeCommand(right,rspeed,forward);
            _delay_ms(10);

    }
    else if(decision = '8') //else case
    {
            lspeed = 25;
            rspeed = 25;//20;

            HBridgeCommand(left,lspeed,forward);
            //_delay_ms(1000);
            HBridgeCommand(right,rspeed,forward);
            _delay_ms(10);

    }
    else if(decision = '9')//detect red line case
    {

            lspeed = 0; //just stop. you're at the red line
            rspeed = 0;//20;

            HBridgeCommand(left,lspeed,forward);
            //_delay_ms(1000);
            HBridgeCommand(right,rspeed,forward);
            _delay_ms(1000);
```

```
                    //up to date

                }

    }
}
```